

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3

10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3

17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I Samyuktha S S(192311056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Systematic Test Design

The team should first identify requirements and design test cases based on them. Techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

can replace ad-hoc testing.

2. Use of Testing Tools

Testing tools can be introduced for test management, automation, defect tracking, and regression testing. Automated tests can repeatedly verify important banking functions such as login, balance enquiry, fund transfer, and transaction validation.

3. Standards and Processes

The team should establish standard procedures for test planning, test-case preparation, execution, defect reporting, and regression testing. Every defect should be recorded with its severity, priority, steps to reproduce, expected result, and actual result.

4. Practical Example

For a fund-transfer feature, test cases should cover:

- Valid account number and sufficient balance
- Invalid account number
- Insufficient balance
- Zero or negative transfer amount
- Transfer exceeding daily limit
- Network interruption during transfer

This provides much better coverage than randomly testing a few transactions.

5. Improvement in Defect Detection

A structured approach ensures that important requirements and boundary conditions are not missed. Regression testing also helps detect defects introduced after software modifications.

Conclusion:

Replacing ad-hoc testing with systematic test-design techniques, appropriate tools, standards, and documented processes makes testing more reliable. It improves defect detection, reduces repeated failures, and increases the overall quality of the banking application.

Q2. Structured Testing Process for an ATM System

Answer:

An ATM system handles important banking operations such as authentication, balance enquiry, cash withdrawal, and transaction processing. Therefore, testing should follow a **structured process throughout the Software Development Life Cycle (SDLC)** instead of being performed randomly. The question specifically requires applying this approach to **ATM withdrawal functionality**.

1. Requirement Analysis

The tester first studies the ATM requirements and identifies what the system should do.

For cash withdrawal, requirements may include:

- The customer must enter a valid PIN.
- The account must have sufficient balance.
- The withdrawal amount must be within the allowed limit.
- The ATM must dispense the correct amount of cash.
- The account balance must be updated correctly.

This stage helps ensure that all requirements are testable.

2. Test Planning

A test plan is prepared by defining:

- Testing objectives
- Scope of testing
- Test cases
- Resources and tools
- Testing schedule
- Expected results

For example, the withdrawal functionality should be tested for both **valid and invalid transactions**.

3. Test Case Design

Test cases are created systematically rather than randomly.

Example test cases:

Test Case	Condition	Expected Result
TC01	Valid PIN and sufficient balance	Cash is dispensed
TC02	Invalid PIN	Transaction is rejected
TC03	Insufficient balance	Withdrawal is rejected
TC04	Amount exceeds withdrawal limit	Transaction is rejected
TC05	Invalid withdrawal amount	Appropriate error is displayed

4. Test Execution

The prepared test cases are executed on the ATM system. The **actual result** is compared with the **expected result**.

For example, if the user requests ₹5,000 and has sufficient balance, the ATM should dispense ₹5,000 and deduct the same amount from the account.

5. Defect Reporting

If the actual result differs from the expected result, the tester reports it as a defect.

For example, if the ATM deducts ₹5,000 but dispenses only ₹4,000, the tester should record the defect with:

- Test case ID
- Steps to reproduce
- Expected result
- Actual result
- Severity and priority

6. Retesting and Regression Testing

After developers fix a defect, the tester performs **retesting** to verify the correction.

Regression testing is then performed to ensure that the modification has not affected other ATM functions such as balance enquiry or mini statements.

7. Test Closure

After testing is completed, the team evaluates the results, documents defects and test outcomes, and determines whether the system satisfies the required quality level.

How Process Consistency Reduces Defects

A defined testing process prevents important scenarios from being missed. Every tester follows the same requirements, test cases, defect-reporting procedures, and validation steps. This reduces inconsistent testing and helps detect defects earlier.

Expected Outcomes

A structured testing process for the ATM system will result in:

- Fewer missed defects
- More consistent testing
- Better test coverage
- Reliable withdrawal transactions
- Faster defect identification and correction
- Improved customer confidence

Conclusion

A structured testing process across the SDLC makes ATM testing systematic and reliable. By following **requirement analysis, test planning, test-case design, execution, defect reporting, retesting, regression testing, and test closure**, the ATM withdrawal function can be thoroughly validated and the number of production failures can be reduced.

Q3. Errors, Defects, and Failures in a Web Application

Answer:

A web application may sometimes produce a result that is different from the expected result. In such situations, it is important to clearly understand the terms **error, defect, and failure**. The given scenario involves mismatches during login and transaction operations.

1. Error

An **error** is a human mistake made during software development. It may occur when a developer misunderstands a requirement or writes incorrect logic.

Example:

A developer misunderstands the login requirement and writes code that accepts an incorrect password under certain conditions.

2. Defect

A **defect** is a flaw in the software caused by an error. It exists in the code or design and may cause incorrect behaviour when executed.

Example:

Suppose the developer writes:

- Expected: Login should be successful only when both username **and** password are correct.
- Actual code: The system checks only the username.

The incorrect authentication logic is a **defect**.

3. Failure

A **failure** occurs when the software actually executes the defective code and produces an incorrect result.

Example:

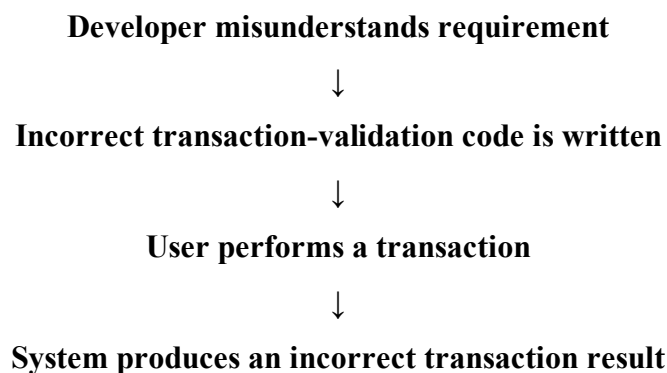
A user enters a correct username but an incorrect password, and the website still allows the user to log in. This incorrect behaviour is a **failure**.

Relationship Between Error, Defect, and Failure

The relationship can be understood as:

Human Error → Defect in Software → Failure during Execution

For example:



Thus, an error may introduce a defect, and the defect can cause a failure when the relevant part of the software is executed.

Example from Login Operation

Consider a login page where the expected behaviour is:

Correct username + correct password → Login successful.

If the developer makes an error while implementing authentication, the resulting code may contain a defect.

For example:

Situation	Expected Result	Actual Result
Correct username + correct password	Login successful	Login successful
Correct username + wrong password	Login rejected	Login successful

The second case demonstrates a **failure caused by a defect** in the authentication logic.

Example from Transaction Operation

Suppose a user has ₹10,000 and transfers ₹3,000.

Expected result:

Account balance should become ₹7,000.

If the system incorrectly deducts ₹4,000, the transaction produces an incorrect output. This is a **failure**, potentially caused by a defect in the transaction-calculation logic.

Effect of Confusing These Terms

Improper understanding can make defect reporting unclear. For example, simply reporting “**login is not working**” does not provide enough information to developers.

A clear defect report should include:

- Defect ID
- Module affected
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshots or relevant evidence

How to Improve Defect Reporting

Testers should clearly distinguish the **cause and effect** of an issue. Instead of writing:

“Transaction has an error.”

A better report would be:

Expected: ₹3,000 should be deducted from the account.

Actual: ₹4,000 was deducted.

Result: Transaction calculation failure.

This makes the issue easier for developers to understand and fix.

Conclusion

Errors, defects, and failures are related but different concepts. An **error** is a human mistake, a **defect** is the resulting flaw in the software, and a **failure** is the incorrect behaviour observed during execution. Clearly distinguishing these terms improves defect reporting, communication between testers and developers, and the overall effectiveness of software testing.

Q4. Defect Clustering and Test Case Prioritization

Answer:

In software testing, defects are often not distributed equally across all modules. A large number of defects may be concentrated in a few modules. This is known as **defect clustering**. The given scenario describes a project where most defects are found in only a few modules, causing testing delays. Therefore, testing should be planned based on defect patterns and risk.

1. Defect Clustering

Defect clustering means that **a small number of modules may contain a large proportion of the defects** in a software system.

For example, consider an online banking application with these modules:

- Login
- Account Management
- Fund Transfer
- Balance Enquiry
- Profile Management

If most defects are repeatedly found in the **Fund Transfer** and **Login** modules, these modules should receive more testing attention.

2. Using Defect Clustering for Test Planning

The testing team can analyze previous defect reports and identify modules with a high number of defects.

Modules can then be classified as:

- **High-risk modules:** Many defects or critical functions
- **Medium-risk modules:** Moderate number of defects
- **Low-risk modules:** Fewer defects and relatively stable functionality

More testers, test cases, and testing time can be assigned to high-risk modules.

3. Test Case Prioritization

Test cases should be executed according to their **risk and importance** rather than simply following their order in the test document.

For example:

Priority	Test Case	Reason
High	Fund transfer with valid details	Critical banking operation
High	Fund transfer with insufficient balance	Important validation
High	Login with invalid password	Security-related
Medium	Balance enquiry	Important but lower risk
Low	Profile information display	Less critical

High-priority test cases should be executed first so that serious defects are identified early.

4. Other Testing Principles

Defect clustering can be combined with other testing principles such as:

Risk-based testing:

Functions that have greater business or security risk should receive more testing.

Early testing:

Testing should begin as early as possible in the SDLC to identify defects before they become expensive to fix.

Exhaustive testing is impossible:

It is not practical to test every possible input and combination. Therefore, testers should focus on important and high-risk areas.

Regression testing:

After fixing defects in a module, previously working functionality should be tested again to ensure that new changes have not introduced additional defects.

5. Practical Example

Suppose an e-commerce application has 1,000 reported defects over several releases. Analysis shows that:

- Payment module → 400 defects
- Login module → 250 defects
- Product search → 150 defects
- Other modules → 200 defects

The testing team should give greater attention to the **Payment and Login modules** because they contain a large proportion of the defects and are also critical to the system.

Test cases related to payment validation, failed transactions, duplicate payments, and incorrect payment amounts should be given high priority.

6. Benefits

Applying defect clustering and test prioritization helps to:

- Detect critical defects earlier
- Reduce testing time
- Improve test effectiveness
- Focus resources on high-risk areas
- Reduce production failures
- Improve overall software quality

Conclusion

Defect clustering helps testers understand where defects are most likely to occur. By analyzing previous defect data and prioritizing high-risk modules and critical test cases, the testing team can use its resources more effectively. This results in **better defect detection, reduced testing delays, and improved software reliability.**

Q5. Defect Classification and Defect Repository

Answer:

A **defect repository** is a structured collection of information about defects identified during software development and testing. In the given scenario, the company already maintains a defect repository but does not use its information effectively, resulting in the same defects appearing in different releases.

1. Origin of Defects

Defects can originate at different stages of software development, such as:

- **Requirement stage:** Incorrect or incomplete understanding of requirements.
- **Design stage:** Incorrect system architecture or design decisions.
- **Coding stage:** Programming or logical errors.
- **Testing stage:** Missing test cases or incomplete test coverage.
- **Maintenance stage:** Changes to existing software introducing new problems.

Understanding where defects originate helps the team identify the root causes.

2. Defect Classification

Defects can be categorized based on their nature and impact.

Common categories include:

- **Functional defects:** A feature does not work according to requirements.
- **Performance defects:** The system responds slowly or cannot handle expected workloads.
- **Usability defects:** The system is difficult or confusing for users.
- **Security defects:** Unauthorized access or improper protection of data.
- **Interface defects:** Incorrect communication between different modules or systems.
- **Logic defects:** Incorrect calculations or decision-making in the program.

Defects can also be classified according to **severity and priority** to determine how urgently they should be fixed.

3. Effective Use of the Defect Repository

The repository should contain important information such as:

- Defect ID
- Defect description
- Module affected
- Defect category
- Severity
- Priority
- Steps to reproduce
- Expected result
- Actual result
- Root cause
- Resolution status

The testing team should regularly analyze this information rather than simply storing it.

4. Using Repository Data for Test Design

Previous defects can be used to create better test cases.

For example, suppose previous releases repeatedly had errors in the **payment module**. The testing team should create additional test cases for:

- Invalid payment details
- Insufficient balance
- Duplicate transactions
- Network interruption
- Incorrect transaction amounts

This ensures that previously problematic areas receive stronger testing in future releases.

5. Preventing Recurring Defects

To prevent the same defects from appearing again, the team should:

1. Analyze the root cause of repeated defects.
2. Update existing test cases.
3. Add regression tests for fixed defects.
4. Improve requirements and design reviews.
5. Use defect trends to identify high-risk modules.
6. Share lessons learned with developers and testers.

Practical Example

Suppose an e-commerce application repeatedly develops an error where customers are charged twice for the same order.

The defect repository shows that similar duplicate-payment defects occurred in three previous releases.

The team can use this information to:

- Add duplicate-payment test cases.
- Perform regression testing after payment-system changes.
- Review the payment-processing logic.
- Identify the root cause.
- Monitor the payment module more closely in future releases.

Expected Quality Improvements

Effective use of the defect repository can result in:

- Fewer recurring defects
- Better test-case coverage
- Faster defect analysis
- Improved root-cause identification
- More effective regression testing
- Better software reliability

Conclusion

A defect repository should not be used only for recording defects. It should also be used as a **source of knowledge for future testing and defect prevention**. By classifying defects, analyzing historical patterns, improving test cases, and addressing root causes, the company can reduce repeated defects and improve the quality of future software releases.

Q6. Identifying the Root Cause of Failures in a Mobile Banking Application

Answer:

A mobile banking application must perform fund transfers accurately and reliably. Frequent crashes during fund transfers may occur because of coding errors, network problems, incorrect input handling, or integration issues. Systematic software testing helps identify the root cause of these failures and prevents them from recurring.

1. Identify the Possible Causes

The tester should investigate the following possible causes:

- Coding defects: Incorrect logic in the fund-transfer function.
- Network failures: The application crashes when the internet connection is interrupted.

- Input validation defects: The system does not properly validate account numbers or transfer amounts.
- Integration defects: The mobile application fails to communicate correctly with the banking server.
- Memory or performance issues: The application crashes under heavy usage or while processing large amounts of data.

2. Apply Systematic Testing

The tester should follow a structured testing process.

Unit testing: Test individual functions, such as balance validation and transaction calculation.

Integration testing: Verify communication between the mobile application, banking server, and database.

Functional testing: Check whether the fund-transfer feature works according to the requirements.

Performance testing: Check whether the application handles multiple transactions without crashing or becoming slow.

Regression testing: After fixing a defect, verify that existing banking functions still work correctly.

3. Design Suitable Test Cases

Test Case	Input or Condition	Expected Result
TC01	Valid account and sufficient balance	Transfer completes successfully
TC02	Insufficient account balance	Transfer is rejected
TC03	Invalid beneficiary account	Appropriate error is displayed
TC04	Internet connection interrupted	Application handles the interruption safely
TC05	User clicks the transfer button repeatedly	Duplicate transactions are prevented
TC06	Server responds slowly	Application handles the delay without crashing

4. Practical Example

Suppose a customer transfers ₹5,000 to another account. The mobile application crashes immediately after the customer clicks the transfer button.

The tester repeats the test under different network conditions and examines application logs and server responses. The investigation reveals that the application does not handle a server timeout correctly.

The developer corrects the error handling, and the tester performs retesting and regression testing. The application should now handle the timeout safely and clearly communicate the transaction status.

The team must also verify that the customer is not charged twice if the transaction was processed before the timeout occurred.

5. Improvements to Prevent Future Failures

- Perform testing under different network conditions.
- Use automated regression tests for fund transfers.
- Validate all user inputs.
- Monitor crash logs and transaction failures.
- Test duplicate requests and interrupted transactions.
- Conduct code reviews and integration testing.

Conclusion

Systematic testing helps identify the root causes of mobile banking failures and ensures that fund transfers are processed accurately. By combining functional, integration, performance, and regression testing, the development team can reduce application crashes, prevent duplicate transactions, and improve banking reliability.

Q7. Difference Between Testing and Debugging

Answer:

Software testing and debugging are two important activities used to improve software quality. Although they are related, they have different purposes. Testing identifies failures, whereas debugging investigates the cause of a defect and helps correct it. Confusing these activities may delay defect resolution and reduce testing effectiveness.

1. Software Testing

Software testing is the process of executing or evaluating software to identify defects and verify whether it meets specified requirements.

The tester compares the expected result with the actual result and reports any mismatch.

Example: A tester enters an incorrect password, but the application allows the user to log in. The tester records this as a defect.

2. Software Debugging

Debugging is the process of locating, analyzing, and correcting the cause of a defect in the software. Developers may examine source code, logs, variables, and execution paths to identify the problem.

Example: The developer investigates the login defect and discovers that the password-validation condition is incorrect. The developer corrects the condition and checks the result.

3. Difference Between Testing and Debugging

Basis	Software Testing	Software Debugging
Purpose	Identifies failures and defects	Finds and fixes the cause of defects
Main activity	Executes test cases and compares results	Examines code and investigates problems
Usually performed by	Testers and developers	Primarily developers
Starting point	Requirements and test cases	A reported or observed problem
Outcome	Test results and defect reports	Corrected code or identified root cause
Example	Detecting incorrect login behaviour	Correcting the login-validation logic

4. Role in Software Quality

Testing helps determine whether the software behaves as expected. Debugging helps developers correct the problems discovered during testing.

Both activities are necessary because finding a defect alone does not guarantee that it will be corrected.

5. Effects of Confusing Testing and Debugging

If testers spend too much time modifying code instead of documenting defects, important issues may remain unreported. Similarly, if developers fix a problem without proper retesting, the correction may be incomplete or may introduce new defects.

This can result in repeated failures, poor communication, and delayed releases.

6. Structured Approach to Integrate Both

1. Testers execute test cases and identify failures.
2. Testers document the steps to reproduce each defect.
3. Developers analyze the defect and identify its root cause.
4. Developers correct the code.
5. Testers perform retesting to verify the fix.
6. Testers conduct regression testing to check existing functionality.

Practical Example

Suppose an ATM deducts ₹2,000 from a customer's account but does not dispense the cash.

The tester reproduces the issue and reports the expected and actual results. The developer investigates the transaction-processing logic and corrects the defect. The tester then verifies the correction and checks related ATM operations.

Conclusion

Testing and debugging are complementary activities. Testing identifies failures, while debugging finds and corrects their causes. Integrating both through clear defect reporting, code correction, retesting, and regression testing improves software quality and reduces recurring failures.

Q8. Cross-Browser Testing for an E-Commerce Platform

Answer:

Cross-browser testing is a software testing technique used to verify that a web application works correctly across different web browsers, operating systems, and screen sizes. An e-commerce platform may behave inconsistently across browsers because of differences in rendering engines, JavaScript support, CSS implementation, or browser settings.

1. Identify the Causes of Defects

Common causes of cross-browser defects include:

- CSS differences: Page layouts and elements may appear differently across browsers.
- JavaScript compatibility: Some functions may not behave consistently across browsers.
- HTML compatibility: Unsupported or incorrectly implemented HTML features may affect the page.
- Browser versions: Older versions may not support newer web technologies.
- Responsive design issues: Pages may not display correctly on different screen sizes.

2. Design a Cross-Browser Testing Strategy

The testing team should prepare a list of browsers and devices commonly used by customers.

The application should be tested on browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, where applicable.

The strategy should include the following activities:

Functional testing: Verify that login, product search, shopping cart, and checkout work correctly.

UI testing: Check the layout, fonts, colours, buttons, and images.

Compatibility testing: Verify that the application behaves consistently across supported browsers and versions.

Responsive testing: Check whether the website adjusts properly to different screen sizes.

Regression testing: After fixing compatibility issues, verify that the changes have not affected other browsers or features.

3. Practical Test Cases

Test Case	Testing Condition	Expected Result
TC01	Open the website in Chrome	Page loads correctly
TC02	Open the website in Firefox	Layout and functions work correctly
TC03	Add a product to the cart in Edge	Product is added successfully
TC04	Complete checkout in Safari	Checkout works correctly
TC05	Open the website on a mobile screen	Layout adjusts to screen size

4. Tools and Techniques

Suitable tools include:

- Selenium: Automates browser-based functional testing.
- Playwright: Supports automated testing across multiple browser engines.
- Browser developer tools: Help identify layout, JavaScript, and console errors.
- Cloud browser-testing platforms: Allow testing on different browsers, operating systems, and devices.

Techniques such as equivalence partitioning, exploratory testing, and risk-based testing can also improve test coverage.

5. How Systematic Testing Improves Reliability

A planned cross-browser testing process identifies compatibility defects before deployment. It ensures that users can browse products, add items to their carts, and complete purchases regardless of which supported browser they use.

For example, if the checkout button works in Chrome but fails in Firefox, cross-browser testing helps identify the issue before customers experience it.

Conclusion

Cross-browser testing is essential for maintaining a consistent user experience on an e-commerce platform. By testing supported browsers, validating layouts and functionality, and using automation tools, the team can identify compatibility defects early and improve website reliability.

Q9. Defect Classification in a Payroll System

Answer:

A payroll system calculates employee salaries based on attendance, working hours, allowances, deductions, taxes, and other payroll rules. Logical errors in the software can produce incorrect salary

calculations, leading to financial losses and employee dissatisfaction. Defect classification helps identify the type, severity, and possible cause of each problem.

1. Types of Defects in a Payroll System

Common defect categories include:

Functional defects: A payroll feature does not meet the specified requirements.

Logical defects: Incorrect conditions or formulas produce wrong results.

Calculation defects: Errors occur while calculating salary, overtime, tax, or deductions.

Data defects: Incorrect employee details or attendance records are processed.

Interface defects: Data is transferred incorrectly between attendance, payroll, and accounting modules.

Performance defects: Payroll processing becomes slow when handling a large number of employees.

2. Classification Based on Severity

Defects can also be classified according to their impact.

- Critical: A major payroll failure affects salary processing for many employees.
- High: An employee's salary is calculated incorrectly because of a significant deduction error.
- Medium: A non-critical payroll report displays incorrect information.
- Low: A minor formatting issue appears on the salary slip.

Severity indicates the impact of the defect, while priority determines how urgently it should be fixed.

3. Practical Example

Suppose an employee earns a basic salary of ₹30,000 and receives an allowance of ₹5,000. The expected gross salary is ₹35,000.

However, the payroll system calculates the gross salary as ₹33,000 because of an incorrect formula.

The tester should:

1. Compare the expected salary with the actual result.
2. Identify the payroll calculation module.
3. Record the incorrect formula result as a defect.
4. Assign an appropriate severity and priority.
5. Report the steps and input data needed to reproduce the issue.
6. Verify the correction through retesting and regression testing.

4. How Defect Classification Improves Testing

Classification helps the team understand which defects require immediate attention and which modules need additional testing.

For example, repeated calculation defects indicate that the salary formula and related validation rules need stronger testing.

The team can design test cases for:

- Basic salary and allowances
- Overtime calculations
- Tax and deduction calculations
- Minimum and maximum salary values
- Employees with missing attendance records
- Employees with different salary structures

5. Preventing Future Defects

The following measures can reduce recurring payroll defects:

- Review payroll formulas before implementation.
- Perform unit testing on individual calculation functions.
- Use independent calculations to verify expected salary values.
- Validate employee and attendance data.
- Conduct integration testing between attendance and payroll modules.
- Maintain a defect repository.
- Perform regression testing after every correction.

Conclusion

Defect classification helps identify and prioritize problems in payroll software. By categorizing logical, functional, calculation, data, and interface defects, the team can design more effective test cases and focus on high-risk areas. Systematic testing and regular regression testing improve salary calculation accuracy and payroll reliability.

Q10. Testing as an Engineering Activity

Answer:

Software testing as an engineering activity means following a systematic, planned, measurable, and documented process to evaluate software quality. Random testing without proper test-case design may miss important defects and produce unreliable results.

In the given scenario, the software team skips structured test-case design and directly performs random tests. The testing process must therefore be redesigned using established testing techniques.

1. Understand the Requirements

The testing team should first study the software requirements and identify the expected behaviour of each feature.

For example, in an online shopping system, requirements may specify that customers must be able to log in, search for products, add items to the cart, and complete payments securely.

Clear requirements help testers determine what needs to be tested.

2. Prepare a Test Plan

A test plan should define:

- Testing objectives and scope
- Features to be tested
- Test resources and tools
- Testing schedule
- Entry and exit criteria
- Risks and priorities

This ensures that testing activities are organized and completed systematically.

3. Design Test Cases

Test cases should be prepared before execution. Each test case should contain the input, execution steps, expected result, and actual result.

Suitable techniques include:

Equivalence Partitioning: Divides inputs into groups expected to behave similarly.

Boundary Value Analysis: Tests values at and around the limits of an input range.

Decision Table Testing: Tests combinations of conditions and their expected outcomes.

State Transition Testing: Verifies behaviour when the software moves from one state to another.

These techniques improve test coverage and reduce the chance of missing important conditions.

4. Use Testing Tools and Standards

Testing tools can help manage test cases, automate repetitive tests, and record defects.

For example, Selenium or Playwright can automate browser-based tests, while defect-tracking tools help developers and testers record and monitor issues.

The team should also follow consistent procedures for test execution, defect reporting, retesting, and regression testing.

5. Practical Example

Consider an online banking application that allows users to transfer money.

Instead of randomly entering amounts, the tester prepares the following test cases:

Test Case	Input or Condition	Expected Result
TC01	Valid beneficiary and sufficient balance	Transfer succeeds
TC02	Insufficient balance	Transfer is rejected
TC03	Transfer amount is zero	Validation message appears
TC04	Transfer amount exceeds the daily limit	Transfer is rejected
TC05	Network interruption during transfer	Transaction status is handled safely
TC06	Repeated transfer request	Duplicate transaction is prevented

The tester executes each case, compares actual results with expected results, and records any defects.

6. How Structured Testing Improves Defect Detection

Structured testing provides several advantages:

- Important requirements are tested systematically.
- Boundary conditions and invalid inputs are considered.
- Test execution becomes repeatable.
- Defects can be reproduced and corrected more easily.
- Test coverage can be measured.
- Regression testing helps prevent previously fixed defects from returning.

7. Improvements to Test Effectiveness

The team should maintain updated test cases, prioritize high-risk features, automate repetitive tests, review test results regularly, and analyze historical defects. Test cases should also be updated whenever requirements change.

Conclusion

Testing should be treated as an engineering activity rather than random execution. By following requirement analysis, test planning, systematic test-case design, tool-assisted execution, defect reporting, and regression testing, the team can improve defect detection and software quality. This approach makes testing more consistent, efficient, and reliable.

Q11. Data Inconsistency in a Healthcare Application

Answer:

A healthcare application handles important patient information such as patient records, medical history, appointments, prescriptions, and billing details. Therefore, data must remain accurate and consistent across all modules. The given problem involves **data inconsistency across modules**, which may occur because of incorrect development, database issues, or improper data handling.

1. Identify the Possible Causes

Data inconsistency may occur because of:

- Incorrect data validation
- Errors in database operations
- Incorrect data synchronization between modules
- Duplicate or missing records
- Incorrect update operations
- Integration defects between modules
- Different modules using different data formats

For example, if the patient's name is updated in the registration module but not updated in the billing module, inconsistent information may be displayed.

2. Apply Systematic Testing

The tester should perform different types of testing:

Functional testing: Verify that each module correctly performs its required operations.

Integration testing: Check whether data is transferred correctly between modules.

Database testing: Verify that data stored in the database matches the information entered by users.

Regression testing: Ensure that corrections do not create new data inconsistencies.

3. Practical Example

Suppose a patient's blood group is recorded as **O+** in the patient registration module but appears as **A+** in the medical-record module.

The tester should:

1. Verify the original input.
2. Check how the data is stored in the database.
3. Check the data transfer between modules.
4. Compare expected and actual values.
5. Report the defect with clear reproduction steps.
6. Retest after the defect is corrected.

4. Defect Tracking and Reporting

Each data inconsistency should be recorded in a defect repository with:

- Defect ID
- Module affected
- Description
- Steps to reproduce

- Expected result
- Actual result
- Severity
- Priority
- Status

This makes it easier to track the defect until it is resolved.

5. Improvements

The organization should:

- Apply proper input validation.
- Use consistent data formats.
- Perform integration and database testing.
- Maintain clear test cases.
- Conduct regular regression testing.
- Analyze repeated defects to identify their root causes.

Conclusion

Systematic testing is essential for maintaining data integrity in healthcare applications. By combining functional, integration, database, and regression testing with proper defect tracking, data inconsistencies can be identified early and prevented from affecting patient-related information.

Q12. Using a Defect Repository to Prevent Repeated Defects

Answer:

A defect repository stores information about defects identified during software development and testing. If a company continues to experience repeated defects despite testing, it indicates that historical defect information is not being effectively analyzed and used for future testing.

1. Analyze Historical Defects

The testing team should review previous defects and identify:

- Frequently affected modules
- Repeated defect types
- Common causes
- Severity and priority
- Defects that appeared after previous changes
- Defects that were reopened

This helps identify patterns in software problems.

2. Use Repository Data for Test Design

Previous defects should be converted into new or improved test cases.

For example, if the payment module repeatedly contains transaction errors, testers should create additional test cases for:

- Invalid payment details
- Duplicate payments
- Failed transactions
- Network interruptions
- Incorrect transaction amounts

These tests should become part of future regression testing.

3. Identify Defect Patterns

Suppose the repository shows that most defects occur in the payment and authentication modules.

The team can classify these modules as **high-risk areas** and give them additional testing effort.

Defect trends can be analyzed based on:

- Module
- Defect category
- Release
- Severity
- Root cause
- Frequency

4. Root-Cause Analysis

The team should not only fix individual defects but also determine **why they repeatedly occur**.

For example, if incorrect calculations repeatedly occur in a billing module, the team should investigate the calculation logic, requirements, and existing test cases.

5. Improvements to Prevent Recurring Defects

The company should:

1. Maintain accurate defect records.
2. Analyze defect trends after every release.
3. Add regression test cases for important defects.
4. Perform root-cause analysis.
5. Review high-risk modules more frequently.
6. Update test cases when defects are discovered.
7. Share lessons learned with developers and testers.

Practical Example

Suppose an online shopping system repeatedly produces an incorrect discount during festival sales.

The defect repository shows that similar discount defects occurred in previous releases.

The testing team can use this information to create test cases for:

- Different discount percentages
- Minimum purchase values
- Maximum discount limits
- Multiple offers
- Boundary values

These cases can then be included in regression testing.

Conclusion

A defect repository is valuable only when its information is actively analyzed and used. By identifying defect patterns, performing root-cause analysis, and converting previous defects into future test cases, organizations can reduce recurring defects and improve software quality.

Q13. Testing a Web-Based Learning System During Peak Usage

Answer:

A web-based learning system may work correctly under normal conditions but fail when many students access it simultaneously. This indicates gaps in **test planning, performance testing, and test coverage**. The testing strategy should therefore include realistic peak-usage scenarios.

1. Identify Testing Gaps

The possible gaps may include:

- No performance testing
- Insufficient load testing
- Testing with only a small number of users
- Lack of peak-usage scenarios
- Incomplete test coverage
- No stress testing
- Insufficient server and database testing

2. Importance of Test Planning

The test plan should identify normal, peak, and extreme usage conditions.

For example, the system should be tested when:

- 100 students are online.

- 500 students are accessing courses.
- A large number of students submit assignments simultaneously.
- Many students take an online examination at the same time.

3. Performance Testing

Performance testing determines how the application behaves under different workloads.

Important measures include:

- Response time
- Throughput
- Resource utilization
- Error rate
- Number of concurrent users

Load testing can be used to check whether the system performs correctly under the expected number of users.

Stress testing can determine how the system behaves beyond its expected capacity.

4. Practical Example

Suppose an online examination system works correctly when 50 students use it. However, when 1,000 students start an examination at the same time, the website becomes slow and some students are unable to submit their answers.

The testing team should reproduce the situation using controlled load testing.

The team can identify whether the problem is related to:

- Server capacity
- Database performance
- Network resources
- Application logic
- Insufficient system configuration

5. Improving Test Coverage

Test scenarios should include:

- Normal usage
- Peak usage
- Multiple simultaneous logins
- Simultaneous assignment submissions
- Online examination
- Large file uploads

- Database-intensive operations
- Network interruptions

6. Expected Improvements

Better performance testing can:

- Detect bottlenecks before deployment.
- Improve response time.
- Reduce application failures.
- Increase reliability during peak periods.
- Improve the user experience.

Conclusion

The failure of a learning system during peak usage indicates insufficient performance and scenario coverage. By including load testing, stress testing, realistic user scenarios, and proper test planning, the organization can identify performance problems before deployment and improve system reliability.

Q14. Early Involvement of Testers in the SDLC

Answer:

Testers should be involved from the **early stages of the Software Development Life Cycle (SDLC)** rather than joining only after development is completed. Early tester involvement helps identify unclear requirements and design problems before they become expensive defects.

1. Role of Testers During Requirement Analysis

Testers can review requirements and identify:

- Ambiguous requirements
- Missing conditions
- Conflicting requirements
- Requirements that cannot be easily tested

For example, instead of a requirement saying “**the system should respond quickly,**” testers can request a measurable response-time requirement.

2. Role During Design

Testers can review the proposed system design and identify possible risks.

They can check whether:

- Important functions are testable.
- Interfaces between modules are clearly defined.

- Error conditions are considered.
- Security and reliability requirements are addressed.

3. Role During Development

Testers can work with developers to understand new features and prepare test cases early.

They can also support:

- Unit-test planning
- Integration-test planning
- Test-data preparation
- Risk identification

4. Role During Testing

Once the software is available, testers execute planned test cases, report defects, perform retesting, and conduct regression testing.

5. Practical Example

Consider a banking application where the requirement states:

“Users should be able to transfer money securely.”

A tester involved early may ask:

- What happens if the account has insufficient balance?
- What is the maximum transfer amount?
- What happens if the network fails during the transaction?
- How are duplicate transactions prevented?

These questions help developers clarify the requirements before implementation.

6. Benefits of Early Tester Involvement

Early involvement helps to:

- Detect defects earlier.
- Reduce the cost of fixing defects.
- Improve requirement quality.
- Increase test coverage.
- Improve communication between developers and testers.
- Reduce misunderstandings.

7. Improving Team Collaboration

The organization should encourage testers, developers, business analysts, and project managers to communicate throughout the SDLC.

Regular requirement reviews, design reviews, and defect discussions can improve collaboration.

Conclusion

Testers should not be considered responsible only for final testing. Their involvement from requirements through development and testing improves software quality and reduces the number of defects reaching later stages. Early collaboration also helps the team build software that is more reliable and testable.

Q15. Improving Test Coverage and Validation

Answer:

Incomplete test coverage occurs when only some possible conditions, inputs, or execution paths are tested. A system may appear to work correctly for normal inputs but fail when it receives unusual or untested inputs. The given scenario requires improving test coverage and test-case design to increase reliability.

1. Identify Coverage Gaps

The testing team should review existing test cases and identify missing areas such as:

- Invalid inputs
- Boundary values
- Error conditions
- Different user roles
- Alternative execution paths
- Integration scenarios
- Exceptional conditions

2. Use Systematic Test-Case Design

Techniques such as the following can improve coverage:

Equivalence Partitioning:

Inputs are divided into valid and invalid groups, and representative values are tested.

Boundary Value Analysis:

Values at the minimum and maximum limits are tested.

Decision Table Testing:

Different combinations of conditions and outcomes are tested.

State Transition Testing:

The system is tested as it moves between different states.

3. Practical Example

Consider a banking system that allows users to transfer between **₹1,000 and ₹50,000**.

Testing only ₹10,000 does not provide sufficient coverage.

Better test cases include:

Test Case	Input	Expected Result
TC01	₹10,000	Transfer accepted
TC02	₹999	Transfer rejected
TC03	₹1,000	Transfer accepted
TC04	₹50,000	Transfer accepted
TC05	₹50,001	Transfer rejected
TC06	Negative amount	Transfer rejected
TC07	Zero amount	Transfer rejected

This tests normal, boundary, and invalid conditions.

4. Importance of Validation

Validation ensures that the software behaves correctly from the user's perspective.

The tester should compare:

Expected Result vs Actual Result

Any difference should be documented and investigated.

5. Improving Test Coverage

The team can improve coverage by:

- Reviewing requirements carefully.
- Mapping test cases to requirements.
- Adding tests for boundary and invalid conditions.
- Prioritizing high-risk features.
- Performing integration and regression testing.
- Reviewing missed defects from previous releases.
- Automating repetitive test cases where appropriate.

6. Benefits

Better test coverage helps to:

- Detect hidden defects.
- Reduce production failures.
- Improve software reliability.
- Increase confidence in the system.
- Ensure important requirements are validated.

Conclusion

Incomplete test coverage can allow defects to remain undetected until deployment. By using systematic test-design techniques, testing boundary and invalid conditions, and reviewing requirements against test cases, the testing team can improve coverage and ensure more reliable software.

Q16. Improving Defect Reporting in a Logistics Application

Answer:

Defect reporting is an important part of software testing because it allows testers and developers to clearly communicate problems. In the given logistics application, poor reporting practices are delaying defect identification and resolution. Therefore, defects should be reported using a **clear and standardized format**.

1. Importance of Clear Defect Definitions

A defect should clearly describe the difference between the **expected result** and the **actual result**.

For example:

Expected: The delivery status should change to "Delivered" after successful delivery.

Actual: The status remains "Out for Delivery".

This clearly explains the problem instead of simply reporting that "delivery is not working."

2. Important Information in a Defect Report

A structured defect report should contain:

- Defect ID
- Defect title
- Module affected
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Test environment
- Screenshots or supporting evidence
- Defect status

3. Practical Example

Suppose a logistics application does not update the delivery status after a package is delivered.

A good defect report would be:

Title: Delivery status not updated after successful delivery

Steps:

1. Open the delivery application.
2. Select an active delivery.
3. Mark the package as delivered.
4. Check the delivery status.

Expected Result: Status should change to "Delivered."

Actual Result: Status remains "Out for Delivery."

This gives the developer enough information to reproduce and investigate the issue.

4. Severity and Priority

Severity describes how seriously the defect affects the system.

Priority indicates how urgently the defect should be fixed.

For example, failure to update a critical delivery status may have **high severity and high priority**.

5. Benefits of Structured Reporting

Clear defect reports help to:

- Reduce communication gaps.
- Make defects easier to reproduce.
- Speed up defect resolution.
- Avoid duplicate defect reports.
- Improve communication between testers and developers.
- Maintain accurate defect history.

Conclusion

Proper defect reporting improves the efficiency of the testing process. By using standardized reports with clear descriptions, reproduction steps, expected and actual results, severity, and priority, the logistics application team can identify and resolve defects more quickly.

Q17. Process-Based Testing for Consistency

Answer:

When a software team follows different testing approaches for different modules, testing becomes inconsistent and important defects may be missed. A **process-based testing approach** provides standardized procedures that all testers can follow.

1. Establish a Standard Testing Process

The organization should define a common process for all modules:

1. Requirement analysis
2. Test planning
3. Test-case design
4. Test environment preparation
5. Test execution
6. Defect reporting
7. Retesting
8. Regression testing
9. Test closure

All testers should follow these steps.

2. Standardize Test Cases

Test cases should use a common format containing:

- Test case ID
- Test objective
- Preconditions
- Test steps
- Test data
- Expected result
- Actual result
- Pass/Fail status

This ensures that test cases created by different testers remain consistent.

3. Standardize Defect Reporting

All modules should use the same defect-reporting format.

For example, every defect should include:

- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Status

4. Practical Example

Suppose an e-commerce system has separate modules for:

- Login
- Product Search
- Shopping Cart
- Payment

If each tester uses a different testing method, some modules may receive detailed testing while others receive only basic testing.

A standardized process ensures that every module is tested systematically using similar documentation and quality criteria.

5. Methods to Ensure Consistency

The team can improve consistency through:

- Standard operating procedures
- Test-case templates
- Common defect-reporting standards
- Test reviews
- Checklists
- Regular team meetings
- Shared testing tools
- Training for testers

6. Benefits

A standardized testing process provides:

- Consistent test execution
- Better test coverage
- Easier defect tracking
- Improved communication
- Reduced testing errors
- Better software quality

Conclusion

Process-based testing ensures that all software modules are tested using a consistent and repeatable approach. By standardizing test cases, execution procedures, defect reporting, and reviews, the organization can improve testing quality and reduce missed defects.

Q18. Boundary Testing for Untested Edge Cases

Answer:

A system may work correctly for normal inputs but fail when users provide values at or beyond the allowed limits. These are known as **boundary or edge cases**. Boundary Value Analysis is an effective testing technique for identifying such defects.

1. Why Edge Cases Are Missed

Edge cases may be missed because:

- Testers focus mainly on normal inputs.
- Requirements are not analyzed carefully.
- Boundary values are not included in test cases.
- Test coverage is incomplete.
- Test cases are created without systematic techniques.

2. Boundary Value Analysis

Boundary Value Analysis focuses on values around the minimum and maximum limits.

For example, suppose an application accepts an age between **18 and 60**.

Important test values are:

- 17 → Below minimum
- 18 → Minimum boundary
- 19 → Just above minimum
- 59 → Just below maximum
- 60 → Maximum boundary
- 61 → Above maximum

3. Practical Example

Consider a banking application where the maximum daily withdrawal is ₹50,000.

The tester should test:

Test Input	Expected Result
₹49,999	Accepted
₹50,000	Accepted
₹50,001	Rejected
₹0	Rejected
Negative amount	Rejected

Testing these values helps identify incorrect validation logic.

4. Including Edge Cases in Test Design

Testers should identify all minimum and maximum limits from the requirements and include:

- Minimum valid value
- Maximum valid value
- Just below minimum
- Just above minimum
- Just below maximum
- Just above maximum
- Invalid values

5. Benefits of Boundary Testing

Boundary testing helps to:

- Detect validation errors.
- Identify incorrect comparison operators.
- Improve test coverage.
- Find defects that normal testing may miss.
- Reduce production failures.

Conclusion

Many software failures occur at the boundaries of allowed input ranges. By using Boundary Value Analysis and deliberately testing values around minimum and maximum limits, testers can identify edge-case defects before deployment and improve system reliability.

Q19. Defect Repository and Documentation

Answer:

Proper defect documentation is essential for tracking, analyzing, and resolving software problems. In the given scenario, the company lacks proper defect documentation, making it difficult to track issues. A structured **defect repository** can solve this problem.

1. Importance of Defect Documentation

Every identified defect should be recorded with sufficient information so that it can be reproduced and resolved.

Important details include:

- Defect ID
- Title
- Description
- Module
- Steps to reproduce

- Expected result
- Actual result
- Severity
- Priority
- Assigned person
- Status
- Resolution

2. Structured Defect Repository

A defect repository acts as a central location for storing all defect information.

It allows the team to track defects through different stages:

New → Assigned → In Progress → Fixed → Retested → Closed

If the defect still exists after testing, it can be reopened.

3. Practical Example

Suppose a shopping application fails during payment.

Instead of simply telling the developer:

"Payment is not working."

The tester records:

Module: Payment

Expected Result: Payment should be completed successfully.

Actual Result: Payment fails after entering valid card details.

Severity: High

Steps to Reproduce: Clearly documented.

This information helps the developer reproduce and fix the issue.

4. Using the Repository for Analysis

The repository can be analyzed to identify:

- Modules with the highest number of defects
- Frequently occurring defect types
- High-severity defects
- Repeated defects
- Defects introduced after changes
- Average time required to resolve defects

This information supports future test planning.

5. Improving Documentation Practices

The company should:

- Use a standard defect-reporting template.
- Assign unique defect IDs.
- Maintain accurate status information.
- Record root causes and resolutions.
- Attach screenshots or logs when necessary.
- Review defect trends regularly.
- Maintain regression tests for important defects.

6. Benefits

A structured repository provides:

- Better defect tracking
- Faster communication
- Easier defect analysis
- Reduced repeated defects
- Better test planning
- Improved software quality

Conclusion

Proper defect documentation makes defect management more systematic and transparent. A well-maintained defect repository helps the team track issues, analyze defect patterns, perform effective regression testing, and prevent similar problems in future releases.

Q20. Improving Testing of a Banking Transaction System

Answer:

Banking transaction systems require high reliability because incorrect transactions can result in financial loss and loss of customer trust. Repeated transaction failures usually indicate weaknesses in the existing testing process. A structured testing approach should therefore be introduced.

1. Analyze the Existing Problems

The team should first identify why transaction failures are occurring.

Possible causes include:

- Incomplete test cases
- Insufficient validation
- Poor integration testing
- Missing edge cases

- Inadequate regression testing
- Incorrect handling of network failures
- Duplicate transaction processing

2. Apply Structured Testing

The testing process should include:

Requirement analysis:

Identify all transaction-related requirements and business rules.

Test planning:

Define the scope, risks, resources, and testing techniques.

Test-case design:

Create test cases for valid, invalid, boundary, and exceptional conditions.

Test execution:

Execute the planned cases and compare expected and actual results.

Defect reporting:

Document failures clearly and assign severity and priority.

Retesting:

Verify that corrected defects have been fixed.

Regression testing:

Ensure that changes have not affected existing transaction functions.

3. Important Transaction Test Cases

Test Case	Condition	Expected Result
TC01	Valid transaction	Transaction completed
TC02	Insufficient balance	Transaction rejected
TC03	Invalid account number	Error displayed
TC04	Amount exceeds transaction limit	Transaction rejected
TC05	Zero or negative amount	Transaction rejected
TC06	Network failure during transaction	Transaction handled safely
TC07	Repeated transaction request	Duplicate transaction prevented

4. Use of Testing Techniques

The team can use:

Equivalence Partitioning:

Test valid and invalid groups of transaction inputs.

Boundary Value Analysis:

Test transaction limits and values immediately above and below them.

Decision Table Testing:

Test combinations such as valid account, sufficient balance, valid amount, and transaction limit.

Regression Testing:

Repeat important transaction tests after software changes.

5. Practical Example

Suppose a customer transfers ₹10,000. The system deducts the amount but the network connection fails before displaying confirmation.

The tester should verify:

- Whether the transaction was actually processed.
- Whether the amount was deducted only once.
- Whether the customer receives the correct transaction status.
- Whether retrying the transaction causes a duplicate transfer.

This type of scenario is important because normal testing may not identify it.

Conclusion

Repeated banking transaction failures can be reduced by replacing unstructured testing with a systematic process. Requirement analysis, structured test-case design, boundary testing, integration testing, defect tracking, retesting, and regression testing help identify defects early and improve the reliability and quality of banking software.